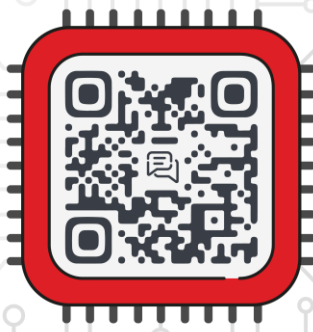




نظري

2

21

2520 sp

كلية الهندسة المعلوماتية

السنة الثانية

Exception handling in Java

د. محمد الأحمد



RB Informatics; 23/10/2024

برمجة ٣

السلام عليكم

في هذه المحاضرة سنتعرف على مفهوم الـ Exception و لذلك سنتعرف على 3 أنواع من الأخطاء التي يواجهها المبرمج

There are 3 types of errors :

1. Syntax error :

❖ هي أخطاء قواعدية تحدث وقت الترجمة (يكتشفها الـ compiler لمعالجتها) ، وهي ناتجة عن عدم الالتزام بقواعد الكتابة (مثل نسيان فاصلة منقوطة / عدم التصريح عن متحول في البداية) وهي من أسهل أنواع الأخطاء .

ملاحظة :

لا يسمح لنا الـ compiler بتنفيذ البرنامج قبل تصحيح الـ Syntax errors (لا يتشكل ملف تنفيذي حتى تصح الأخطاء)

2. Runtime error :

❖ هي أخطاء تحدث في وقت التنفيذ وتسبب توقف البرنامج ، وهي تعتبر Exception (تعمل crash للبرنامج) .

■ مثال : قسمة عددين حيث العدد الثاني هو الصفر ، و أيضاً فتح ملف غير موجود ، ومحاولة الوصول إلى عنصر خارج حدود المصفوفة ، و التعامل مع ob وهو null ، و هي تعالج بما يسمى الـ exception handling .

3. Logical error :

❖ وهي تحدث عندما نحصل على نتيجة غير متوقعة (ليست صحيحة) نتيجة عدم فهم المبرمج للخوارزمية، أي سببه المبرمج ، و تعالج بالـ software testing .

الـ software testing هي مرحلة من مراحل تطوير البرنامج

■ ملاحظة أربيسيزية :

ما الفرق بين :

Error

هي مشكلة يصعب حلها جداً

- خارج سيطرة المبرمج

(environment)

من بيئة التنفيذ

Exception

هي مشكلة يمكن إيجاد حل لها

مثل : file / network

ملاحظات :

• runtime error غير مسؤول عنها الـ compiler

Exception : an event cause the execution to stop

• المبرمج هو الذي يتعامل مع الـ exception

في حال ظهور exception ووجود العديد من التعليمات بعد التعليمة التي سببت الـ exception فإنها لا تنفذ بسبب توقف البرنامج ، و تعالج هذه الأخطاء عن طريق الـ

• Exception handling:

هي آلية للتعامل مع الـ exception بشكل أساسي كي لا يتوقف البرنامج بشكل كامل (نعلم بحصول الخطأ ولكننا نريد تجاوزه وإكمال تنفيذ البرنامج)

• لمعالجة الـ exception هناك كلمات مفتاحية وهي :

(1 Try (2 catch (3 finally (4 throw (5 throws

ستتعرف عليهم في هذه المحاضرة

• عند ظهور الـ exception يظهر ما يلي :

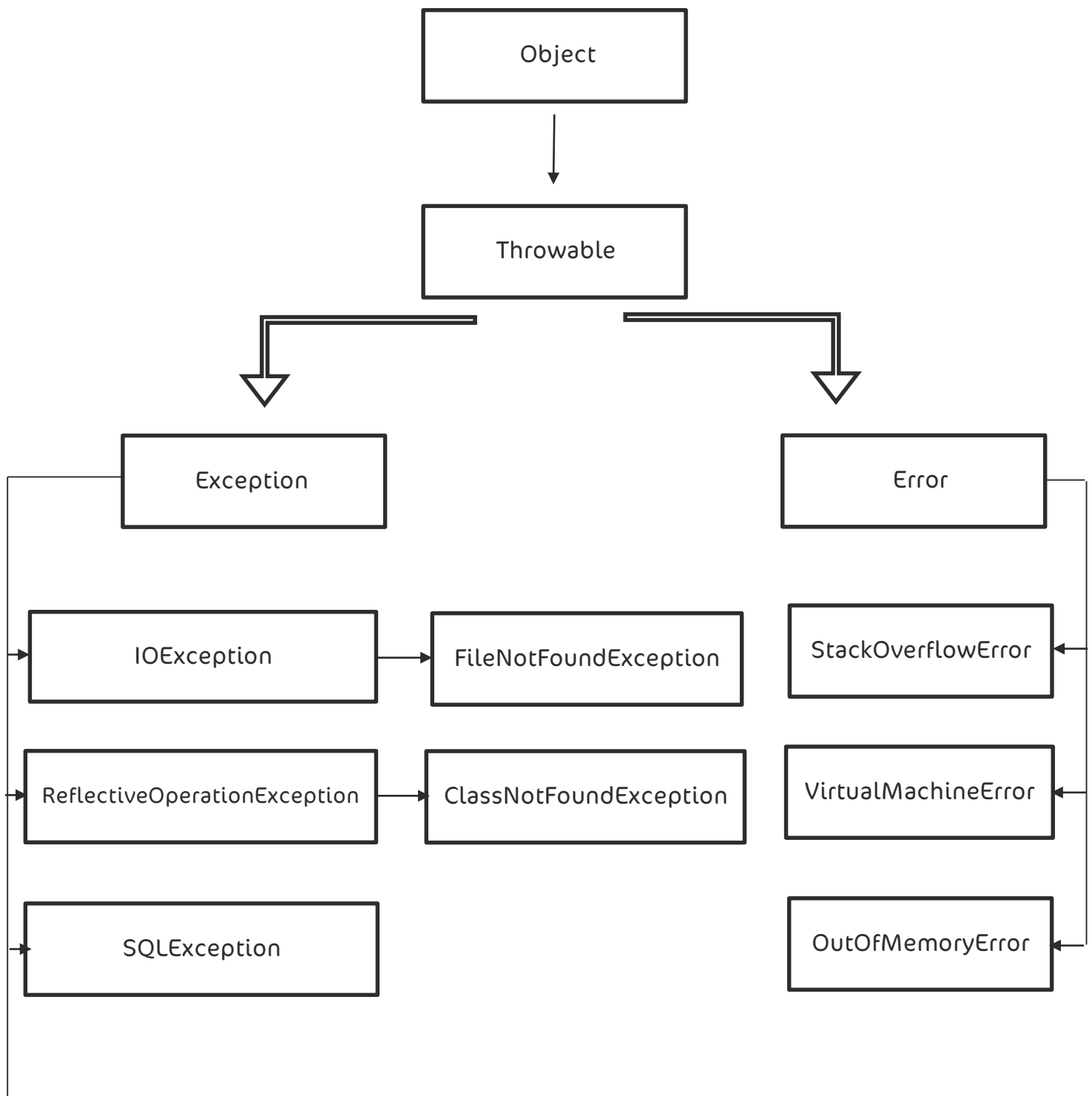
Exception type : description

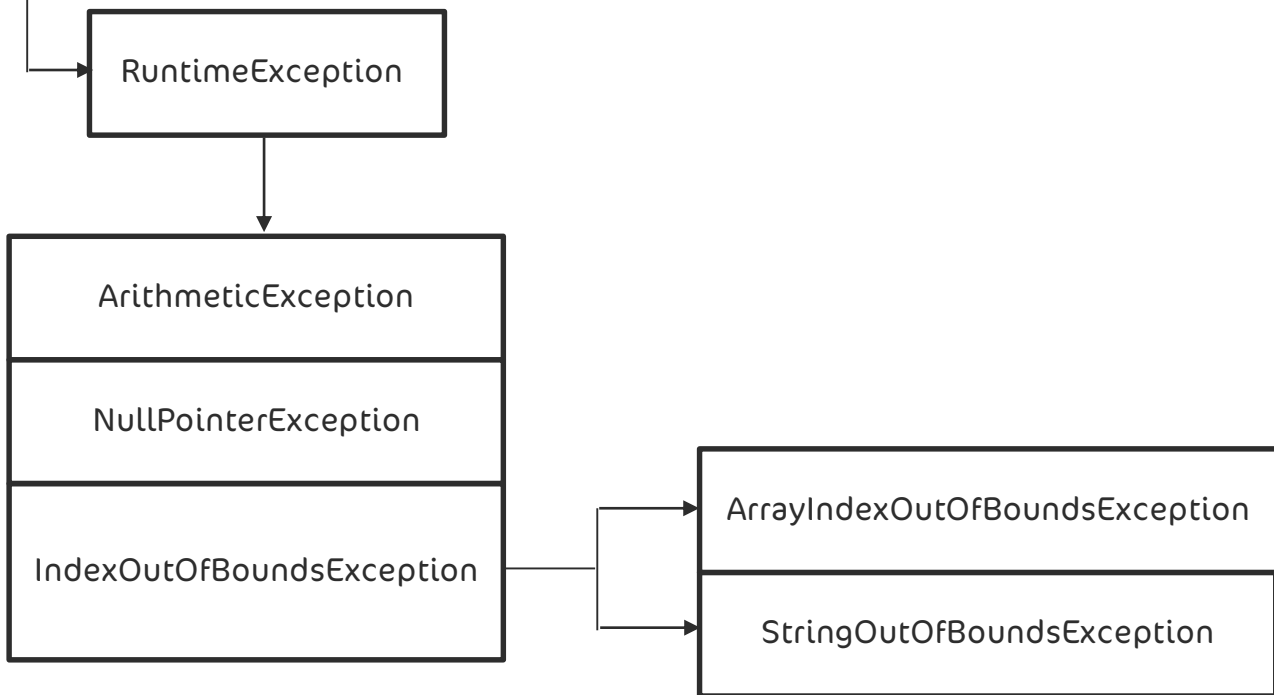
وصف الخطأ اسم الخطأ

• كل exception هو object لـ class معرف مسبقاً

هرمية ال Exception

في الجافا نعلم أن الأب الروحي هو object وهو الكلاس الأب الذي ترث منه جميع الكلاسات وأيضاً ال throwable class يرث من ال object class ولكن ال throwable class هو الأب لكل ال exceptions فهي جميعها ترث منه





ملاحظة: يمكن حفظ هذه الهرمية و سنتعلم فيما بعد أهمية هذه الهرمية

ملاحظة ارييسيزية :

الكلاس الأب throwable يحوي على العديد من ال methods مثل :

- getMessage : تعطينا رسالة تبين السبب الذي من أجله حصل ال exception أي (الحصول على وصف نصي للخطأ)
- toString : تقوم بإرجاع اسم الكلاس الخاص بالاستثناء مع رسالة الخطأ إن وجدت
- printStackTrace : هذه الميثود تقوم بطباعة اسم الاستثناء و رسالة الخطأ و تتبع الخطأ مما يساعد على فهم مكان حدوث ال exception

يحتوي ال Exception class على بانني بدون قيم و بانني مع قيم من الممكن أن يستقبل message ، و بانني آخر من الممكن أن يستقبل عدة باراميترات، ولكنه لا يحوي أي ميثود خاصة به بل يرثها جميعها من كلاس Throwable.

لن يحمك الله ما لا تطيقه

لذلك

انت دائماً تستطيع..

Types of java exception

لدينا نوعان من الـ exception هما :

checked

الـ compiler يجبرنا على معالجة هذه
الـ exceptions و التي تكون

- IOException
- SQLException
- ClassNotFoundException

- أي جميع الكلاسات الأخرى ما عدا class الـ
RuntimeException و أبنائه
- أي عند محاولتنا فتح ملف يجبرنا الـ
compiler عمل handling الـ Exception

Unchecked

كل الـ classes التي ترث من الكلاس

unchecked هي RuntimeException
فهي لا تكتشف إلا عند التنفيذ لذلك
الـ compiler لا يتوقع حدوث exception

■ أمثلة اربيسيزية على أنواع الـ Exception :

1. Array Index out of bounds Exception

مثال :

```
int arr [] =new int [3];  
arr [5]=10 ;
```



في هذا السطر يعطي Array Index of Bounds Exception
لأنه لا يمكن الوصول إلى الـ Index الخامس وهو خارج حدود المصفوفة

2. NullPointerException

❖ يحدث عندما نريد الوصول إلى Value و لكن قيمة المتغير الخاص بها هي null

مثال:

```
int [] arr = null ;
System.out.print(arr[1]);
```

لذلك يحصل Exception

3. NumberFormatException

مثال:

```
String a = "java";
int n =Integer.parseInt (a);
```

يتم رمي NumberFormatException وذلك لأن قيمة الـ String غير قابلة للتحويل إلى عدد صحيح.
فائدة : الدالة Integer.parseInt() في Java تُستخدم لتحويل سلسلة نصية تمثل عدداً صحيحاً إلى قيمة صحيحة من نوع int.

4. ArithmeticException

مثل القسمة على صفر :

```
int x =5/0;
System.out.print(x);
```

سيعطينا Exception لكن إذا كانت المتغيرات من نوع double فعندها سوف تكون النتيجة ∞

ملاحظة: أخطاء الـ null pointer في الجافا قليلة بسبب وجود garbage collector.

Garbage collector in Java

- تُستخدم لإدارة الذاكرة تلقائياً حيث يقوم الـ GC بتحرير الذاكرة المستخدمة بواسطة الكائنات التي لم تعد بحاجة إليها.
- يتم تشغيل الـ GC في أوقات معينة (عندما تكون الذاكرة منخفضة) أو بشكل دوري.

Garbage collector in C++

- يجب على المبرمجين إدارة الذاكرة يدوياً باستخدام new و delete لإنشاء وتحرير الكائنات، ولا يوجد GC تلقائي في اللغات الأقل ديناميكية من جافا كـ C++.
- تحرير الذاكرة هي مسؤولية المبرمج في C++.

```

1 import java.util.Scanner;
2 class Test {
3     public static void main (String [] args){
4         int a,b,c,d;
5         Scanner in = new Scanner(System.in);
6         a=in.nextInt();
7         b=in.nextInt();
8         c=a/b;
9         d=a+b;
10        System.out.println(c);
11        System.out.println(d);
12    }
13 }

```

- ❖ في هذا المثال يتم تعريف متحولين صحيحين a,b و يتم إدخالهم من لوحة المفاتيح
- ❖ عند إدخال 4 و 2 سيكون الـ output هو 2 و 6
- ❖ عند إدخال المحرف 'a' يعطينا Exception و يتوقف البرنامج و تظهر الرسالة التالية :

Exception in thread "main"
 java.util.InputMismatchException
 وهو خطأ عدم مطابقة الدخل لأن الدخل يجب
 أن يكون من نوع int

- ❖ عند إدخال 5 و 0 يعطينا أيضاً Exception على الرغم من أن نوع الدخل ليس خاطئاً ولكن هناك قسمة على صفر و تظهر الرسالة :

Exception in thread "main" —————→ الخطأ ظهر في الـ main
 Java.lang.ArithmeticException : —————→ اسم الخطأ
 / by zero —————→ الوصف

هناك خمس كلمات مفتاحية لمعالجة الـ Exception وهي :

1. Try (جرب)

- هي Block نضع فيها الـ statement التي نتوقع أن يحدث فيها Exception خلال الـ Runtime

```
try {
    statement;
}
```

- من الممكن أن نجد try أولى و بداخلها أيضاً try أخرى
- من غير الممكن أن نجد try لوحدها من دون إحدى الـ keyword التالية :

1. Catch

2. Finally

2. catch

- هي Block ثاني بعد الـ try و تتيح لنا معالجة الخطأ بحسب نوعه من خلال تمرير نوع الـ Exception و نضع فيها التعليمات التي نعالج فيها الخطأ بحالة ظهوره

```
catch (Exception type obj){
}

```

الـ class الخاص بنوع الـ Exception

خلق obj من كلاس الـ Exception

- من الممكن وجود أكثر من catch بحيث تكون كل catch تعالج خطأ معين
- بالعودة إلى المثال السابق نلاحظ أنه من الممكن أن نواجه هذه Exceptions: `InputMismatchException` 1.

مثل عند إدخال كلمة Mohamad سوف يتوقف البرنامج و يعطينا Exception لعدم مطابقة نوع الدخل لنوع المتغير a.

عند إدخال a, b و كان b=0 فيعطينا Exception لأن القسمة على صفر غير منطقية .

- و لمعالجة هذه المشاكل نستخدم الـ try و catch في المثال السابق

المثال السابق :

```

1 import java.util.InputMismatchException;
2 import java.util.Scanner;
3 class Test {
4     public static void main (String [] args){
5         int a,b,c,d;
6         Scanner in = new Scanner(System.in);
7
8         try {
9             a=in.nextInt();
10            b=in.nextInt();
11            c=a/b;
12            d=a+b;
13            System.out.print(c);
14            System.out.print(d);
15        }
16        catch(InputMismatchException ie){
17            System.out.println(ie);
18        }
19
20    }
21 }

```

الشرح:

عرفنا 4 متغيرات a, b, c, d و وضعنا الـ statement التي نتوقع أنها تسبب exception في try Block لمعالجته حيث عند إدخال أي حرف أو String أو أي دخل لا يطابق نوع الـ int من لوحة المفاتيح يتم خلق object من الـ exception ، ثم البحث عن catch تحمل نفس نوع الـ Exception وإرساله ليتم معالجته ، وفي حال عدم إيجاد catch لن يعالج وستظهر على الشاشة رسالة الـ Exception دون معالجة و يتوقف البرنامج.

Input : mohamed

Output :

java.util.InputMismatchException

■ ملاحظة :

(1) في حال كتبنا في catch Block

■ System.out.print(ie);

أو

■ System.out.print(ie.toString());

سيتم طباعة نفس ال message لأن ال object تلقائياً يستدعي هذه الميثود لطباعة رسالة الخطأ.

(2) يمكن إضافة هذه التعليمة أيضاً في ب catch Block لطباعة رسالة مخصصة بالخطأ:

■ System.out.print(ie.getMessage());

و لطباعة مصدر الخطأ أيضاً يمكن إضافة:

■ ie.printStackTrace();

تتمة المثال السابق:

في حال كان نوع الدخل صحيحاً ولكن قيمة b تساوي الصفر أي $a=5$ ، $b=0$ ← Arithmetic Exception

فإن ال catch التي وضعناها لا تحمل نفس نوع الخطأ فلن يتم معالجة ال Exception

لذلك لا بد من إضافة catch جديدة تناسب نوع ال Exception

```
catch(ArithmeticException ae){
    System.out.print(ae);
}
```

و بذلك تتم معالجة ال exception و ظهور الرسالة :

java.lang.ArithmeticException: / by zero

■ نتائج :

1. يمكن وضع أكثر من statement ضمن ال try

2. يتم تنفيذ ال catch فقط عند ظهور ال Exception المناسبة لها

3. بعد ظهور ال Exception فإن التعليمات داخل ال try لا تنفذ (حتى ولو كانت صحيحة) لذا يجب وضع التعليمات المتوقعة أن تسبب Exception فقط في ال try.

مثل: في try Block عند إدخال $b=0$ ، $a=5$ فإن التعليمة $c=a/b$ سوف تسبب Exception في هذا السطر و لن

يتم تنفيذ التعليمة التي بعدها $d=a+b$; على الرغم من أنها صحيحة

ملاحظات :

- فائدة الـ try هي عزل الأخطاء عن الكود و عن الـ Debugging
- في حال عدم ظهور الـ Exception لا يتم تنفيذ الـ catch و سيتم تنفيذ التعليمات normally
- من الممكن أن نضع try و catch داخل الـ do while

➤ في هذا المثال سنوضح استخدام أكثر من catch :

```

1 class Test{
2     public static void main(String[] args){
3         try {
4             int arr[] =new int[5] ;
5             arr[10]=7/0;
6             System.out.print("the value is:");
7         }
8         catch
9 (ArrayIndexOutOfBoundsException ex){
10             System.out.print(ex);
11         }
12         catch(ArithmeticException ex){
13             System.out.print(ex);
14         }
15     }
16 }
```

في هذه الحالة يظهر لدينا Exception و لكن السؤال

ما هو نوع هذا الـ Exception ؟

و أي الـ catch نفذت أولاً ؟

الجواب :

سيظهر Arithmetic Exception ، لأن عملية الإسناد تبدأ من اليمين إلى اليسار، أي إن أول Exception حصل نوعه Arithmetic بسبب القسمة على صفر و التقطته الـ catch الثانية المطابقة لنوعه .



ملاحظة ارييسيزية :

- يمكن تمرير أكثر من Exception ضمن ال catch و ذلك عن طريق (|) ← معناها or وهذا ما يسمى بال union catch و تم تضمينها بالجافا بعد الإصدار السابع أي يمكننا في المثال السابق استبدال ال catch ب catch واحدة فنكتب:

```
catch(ArrayIndexOutOfBoundsException | ArithmeticException ex){
    System.out.print(ex);
}
```

و ستظهر نفس النتيجة

ملاحظة: في حال كتابة catch من نوع الأب Exception ماذا سوف يحصل :

في المثال السابق :

```
catch(Exception ex){
    System.out.print(ex);
}
catch(ArithmeticException ex){
    System.out.print(ex);
}
catch(ArrayIndexOutOfBoundsException ex){
    System.out.print(ex);
}
```

- سوف يظهر لدينا compiler error أي أن البرنامج لا يترجم
- فعند كتابة catch(Exception) سيتم التقاط جميع الأخطاء التي ترث من class Exception فالأب يشمل الأبناء و يعبر عنهم، لذلك يجب الانتباه في استخدام أكثر من catch والترتيب من الأبناء إلى الآباء.
- فلا يجوز أن يسبق كلاس الأب كلاس ابنه super → sub

ملاحظة : من الممكن كتابة فقط catch نوع الأب Exception لوحدها لأنها تحوي على جميع الأبناء ولكن هذا غير مستحسن.

مثل المثال السابق :

```
Catch (Exception ex){
    System.out.print(ex);
}
```

3. Finally

- هذه الـ keyword أيضاً لمعالجة الأخطاء ، مثل الـ catch ولكن finally تنفذ دوماً
- الهدف منها تحرير المصادر بشكل آمن
- في حال عدم ظهور Exception تنفذ finally أيضاً
- من الممكن أن تأتي مع أكثر من catch و تنفذ في حال تم تنفيذ الـ catch أم لا
- مثال عن الـ finally : في حال محاولة فتح ملف ونريد التأكد إذا كان مغلقاً أم لا ، إغلاق المصادر الخارجية
- حول المثال في الصفحة التاسعة:

```
Scanner in = new Scanner(System.in);
int a,b,c,d;
a=in.nextInt();
b=in.nextInt();

try{
    c=a/b;
    System.out.println(c);
}
catch(ArithmeticException ae){
    System.out.println(ae);
}
finally{
    d=a+b;
    System.out.println(d);
}
```

الشرح:

تتم معالجة الـ Exception ويتم تنفيذ الـ finally و طباعة 5

- Input : 5 0
- Output:
java.lang.ArithmeticException: / by zero
5

ملاحظة ارييسيزية :

- في حال تم وضع try و Finally فقط سيتم تنفيذ ال finally ومن ثم يعطينا Exception في حال وجوده

مثل: لو وضعنا في المثال السابق:

```
try{
    c=a/b;
    System.out.println(c);
}
finally{
    d=a+b;
    System.out.println(d);
}
```

➤ Input : 5 0

➤ Output: 5

Exception in thread "main" java.lang.ArithmeticException : / by zero

ملاحظات حول ال try , catch , finally

1. كل try تحتاج إلى catch واحدة على الأقل أو finally
2. يمكن أن نلحق try بـ catch و finally معاً
3. يمكن أن نلحق try بأكثر من catch ومعهم finally أيضاً
4. العملية التي تحدث في try block في حال ظهور exception هي throwing له، وبدونها لن يحدث catching للexception.

ملاحظة ارييسيزية :

- هناك حالتان فقط لا تنفذ فيهما ال finally و هما :
(1) في حال وضعنا :

```
System.exit(0);
```

- أي معنى هذه التعليمة الخروج من البرنامج
- (2) في حال fetal error ولكن لن نتطرق كثيراً في هذا الموضوع

- Keyword used to create a custom Exception or error

- أي تستخدم لإلقاء Exception بشكل صريح ، و تستخدم داخل ميثود ، و بما أن كل Exception هو object لنولد كائنًا من هذا النوع نستخدم new

```
throw new Exception();
```

- نستطيع استخدام الـ throw داخل الـ catch ايضاً

ملاحظة ارييسيزية :

- يمكن خلق object بالطريقة التي اعتدناها حيث :

```
ArithmeticException ae = new ArithmeticException ();  
throw ae;
```

ولكن ما الفرق بين الطريقة الأولى و الثانية ؟
الفرق أنه في الطريقة الثانية سوف يحتفظ بقيمة الـ object داخل reference ، ويمكن إعادة استخدامه أو إضافة رسالة مخصصة له.

- throw تستخدم لإلقاء Exception واحد فقط
- عند خلق object :

```
throw new Exception type();
```

سوف تكون الرسالة التي تظهر هي الرسالة الأصلية ويمكن تغييرها باستخدام الباني مع قيم وتمرير الرسالة كبرامير فنكتب :

```
throw new Exception ("Message")
```

- عند استخدام throw يخبرنا الـ compiler بضرورة معالجة هذه الـ Exception فعندما نجد الـ throw داخل Method أو ضمن try يجب أن نجد catch مناسبة لمعالجة الـ Exception
- وفي هذا المثال سيتضح استخدام الـ throw :

```
public class main{
    public static void validate(int Number)
    {
        if(Number<0)
            throw new ArithmeticException();
        else
            System.out.print("message");
    }
    public static void main (String[] args){
        validate(-1);
    }
}
```

■ الشرح:

- في هذا المثال لدينا validate method تستقبل باراميتر ، و تم خلق كائن من throw للإلقاء Exception
 - وعند تمرير القيمة (-1) إلى validate method سيتم إلقاء ArithmeticException لذلك يخبرنا ال compiler به ، ولمعالجة هذا ال Exception يمكننا وضع ال keyword (catch, try) بطريقتين:
1. في المكان الذي تم فيه استدعاء هذه الميثود
 2. في مكان تعريف الميثود نضع try و catch

■ الطريقة الأولى :

```
1 public class main {
2     public static void validate(int number)
3     {
4         try{
5             if(number<0)
6                 throw new ArithmeticException();
7
8             System.out. print("no Exception");
9
10        }
11        catch(ArithmeticException ae){
12            System.out.print(ae);
13        }
14    }
15    public static void main(String[] args) {
16        validate(-1);
17    }
18 }
```

output : java.lang.ArithmeticException

2. الطريقة الثانية :

(بداية البرنامج نفسه)
بتابع الـ main :

```
try{
    validate(-1);
}
catch(ArithmeticException ae){
    System.out.print(ae);
}
```

output : java.lang.ArithmeticException

ملاحظة: استخدام الطريقة الثانية أفضل عند وجود العديد من الـ methods, فنقوم بـ handling الـ method فقط عند استدعائها.

- تستخدم للتصريح عن الـ Exception ، نكتب بعدها الـ Exception type ونستخدم مع الميثود عن طريق وضعها في method signature قبل الـ body ولكنها لا تلقي Exception ، وتدل على أن هذه الميثود قد تقوم بإلقاء خطأ من النوع الذي ذكر في الـ throws
- تستخدم عندما لا نريد معالجة الـ Exception في مكان وقوعه فنترك المعالجة للـ caller.
- الـ caller : هو المكان الذي تستدعى فيه الميثود الذي يحوي الـ signature الخاص بها على throws (قد يكون الـ main مثلاً).

مثال :

```

1 import java.io.*;
2 class Main{
3
4 void method() throws IOException{
5     throw new IOException("device error");
6 }
7
8 public static void main(String[] args) {
9     try {
10         Main m = new Main();
11         m.method();
12     } catch (Exception e) {
13         System.out.print("Exception ");
14     }
15     System.out.print("normal flow");
16 }
17 }

```

Output: Exception normal flow

■ شرح المثال :

قمنا بتعريف الـ method التي قد تلقي Exception وتم استدعاؤها في الـ main ، فهنا تقوم الـ method بالتنفيذ وفي حال تم إيجاد IOException تقوم بإلقائه إلى catch وتقوم بالمعالجة.

■ ملاحظات عن throws :

Unchecked Exception	Checked Exception
التصريح عنه بـ throws خطوة اختيارية.	إن لم يُعالج هذا النوع من الـ exceptions بـ try و catch فحسراً يجب التصريح عنه بـ throws في توقيع الدالة.

■ Custom Exception

نلاحظ أنه في java Exception

Custom Exception (User defined Exception) ←

Build Exception ←

● وسنتكلم في هذه الفقرة عن Custom Exception :

- يقوم المبرمج بتعريف الـ Exception الخاص به لتلبية متطلبات أو لمعالجة حالات خاصة في برنامجه، ويتم استخدامها عندما لا تكون الـ Exceptions الجاهزة التي توفرها Java كافية للتعامل مع الـ Exceptions في البرنامج الخاص بنا.
- يكون التعريف مثل أي كلاس ، و من أجل أن يكون من نوع Exception يجب أن يرث من class نوعه Exception معرف مسبقاً ، و في حال ورث من Checked Exception سيحصل على صفاته.

```
class InvalidException extends Exception {
    public InvalidException(String str) {
        super(str);
    }
}

public class TestCustomException {
    static void validate(int age) throws InvalidException {
        if(age<18){
            throw new InvalidException ("age is not valid to vote ");
        }
        else
            System.out.print("welcome to vote ");
    }
    public static void main(String[] args) {
        try{
            validate(13);
        }
        catch( InvalidException ex){
            System.out.println("caught the Exception");
            System.out.println(ex);
        }
    }
}
```

لا تستسلم ابداً الأشياء العظيمة تأخذ وقت

الشرح :

في هذا المثال تم تعريف class InvalidAgeException وهو custom Exception ، وورث من كلاس الأب وفيه ميثود تفحص شرطاً معيناً ، وتم استخدام الـ throw و throws فيها بحيث يتم فحص العمر و إذا تحقق الشرط يتم إلقاء Exception مع رسالة معينة ، و إلا فيتم طباعة welcome to vote ، وعند التنفيذ تم إلقاء InvalidAgeException وتمت معالجته باستخدام catch مناسبة

Output :

Caught the Exception

exceptions.InvalidAgeException: age is not valid to vote

الهدف من عملية Exception handling

1. تنفيذ البرنامج كاملاً.
2. إظهار رسائل مناسبة للمستخدم تعبر عن نوع الخطأ.
3. تحرير المصادر.

السليبات :

Expensive ولذلك طالما أمكن الاستغناء عنها فهو أفضل .

فمثلاً : أن نستبدلها بـ if و else في بعض الأوقات يجعل الكود أسرع وتحل المشكلة ، مثل لو كان هناك String وهو null يمكننا كتابته باستخدام if.

```
public class Main {
    public static void main(String[] args) {
        String a = null;
        Try {
            System.out.println( a.length());
        }
        catch (NullPointerException e) {
            // التعامل مع الاستثناء
            System.out.println("a is null");
        }
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        String a = null;
        if (a == null) {
            System.out.println("a is null");
        } else if (a.isEmpty()) {
            System.out.println("a is empty");
        } else {
            System.out.println("the value of a is " + a);
        }
    }
}
```

في الكود الأول تم استخدام جملة if-else ، وهو أكثر كفاءة من حيث الأداء لأنه يتجنب رمي Exception.

أما في الكود الثاني سيتم رمي NullPointerException ، وفي حال وجود العديد من القيم التي تكون null سيؤدي إلى استثناءات متكررة.

The end